

SYS_CTRL states:

IDLE	= 4'b0000 = 0
REG_WR_1	= 4'b0001 = 1 received first wr frame, waiting for second
REG_WR_2	= 4'b0011 = 3 received second wr frame, waiting for third
REG_WR_3	= 4'b0010 = 2 received third wr frame, writing in reg file... (1 cycle)
REG_RD_1	= 4'b0110 = 6 received first rd frame, waiting for second
REG_RD_2	= 4'b0111 = 7 received second rd frame, waiting for rd_valid
REG_RD_RES	= 4'b0101 = 5 received rd_data from reg_file, writing in FIFO... (1 cycle)
ALU_OP_1	= 4'b0100 = 4 received first ALU frame, waiting for second
ALU_OP_2	= 4'b1100 = C received second ALU frame, waiting for third
ALU_OP_3	= 4'b1101 = D received third ALU frame, waiting for fourth
ALU_OP_4	= 4'b1111 = F received fourth ALU frame, waiting for ALU_out_valid
ALU_OP_RES	= 4'b1110 = E received ALU_OUT, writing in FIFO... (1 cycle)
ALU_NO_OP_1	= 4'b1010 = A received first ALU_NO_OP frame, waiting for second
ALU_NO_OP_2	= 4'b1011 = B received second ALU_NO_OP frame, waiting for ALU_out_valid
ALU_NO_OP_RES	= 4'b1001 = 9 received ALU_OUT, writing in FIFO... (1 cycle)
RESULT_2	= 4'b1000; //8 :this state for multiplication results i.e.writing the second byte of ALU result in the fifo

TX states:

IDLE	= 3'b000 = 0
START	= 3'b001 = 1
DATA	= 3'b010 = 2
PARITY	= 3'b011 = 3
STOP	= 3'b100 = 4

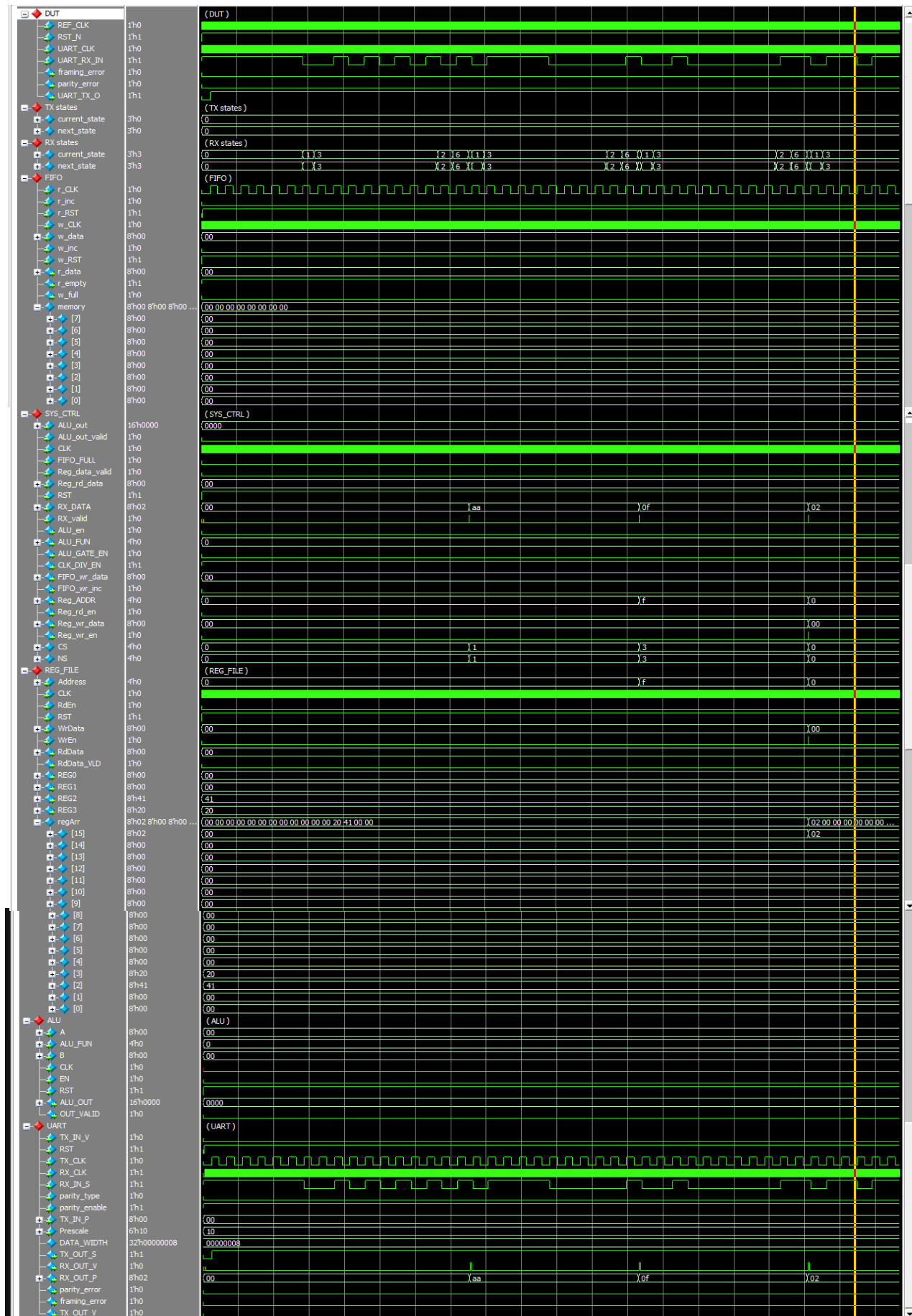
RX states:

IDLE	= 3'b000 = 0
START	= 3'b001 = 1
DATA	= 3'b011 = 3
PARITY	= 3'b010 = 2
STOP	= 3'b110 = 6

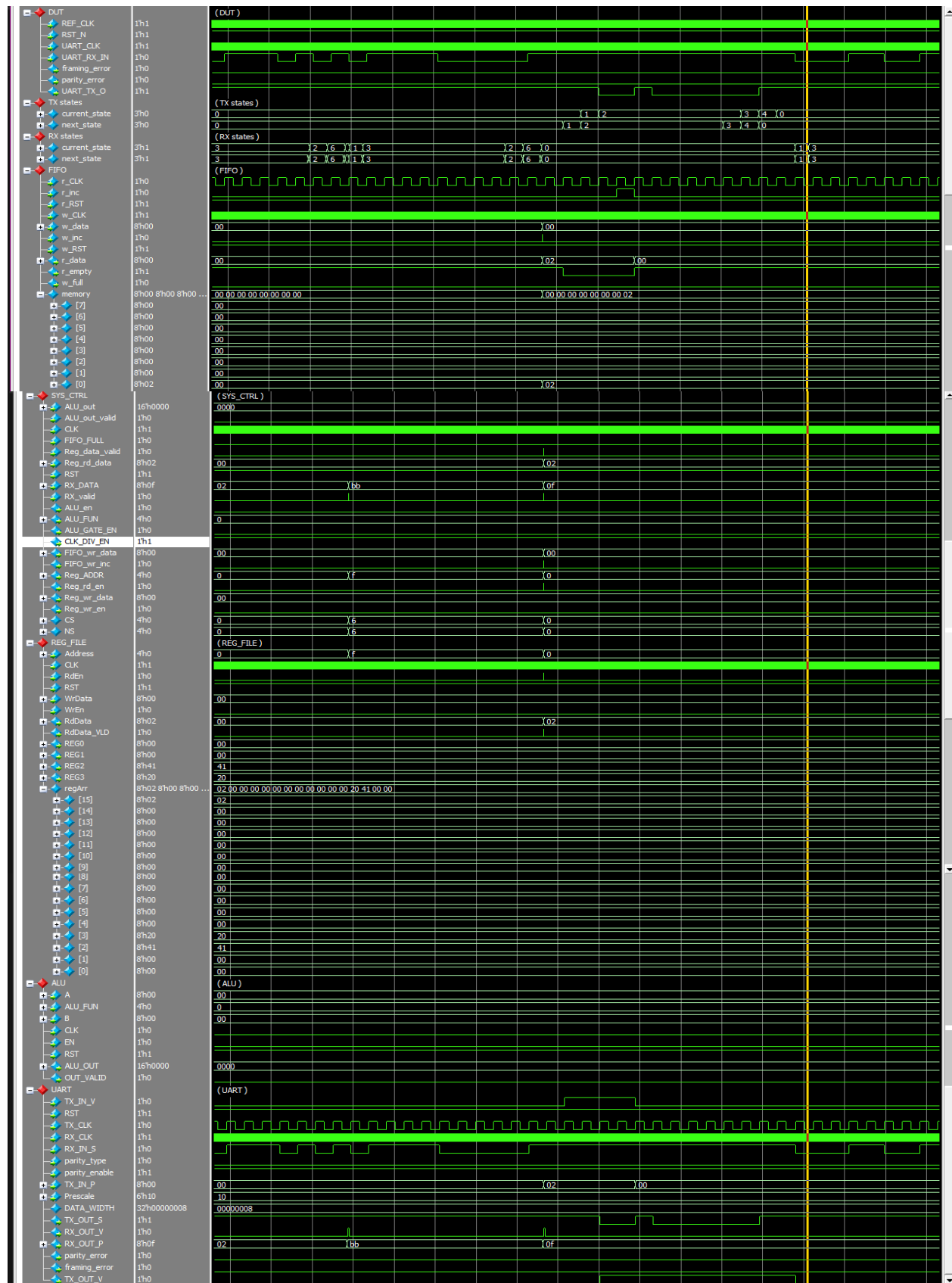
Sequence of operation of TB:

START => Checking the current PRESCALE => Writing 02 in address F of reg file => reading address F of reg file => adding 5 and 6 => multiplying 64 and 7 => subtracting with no operands (i.e. 64 - 7) => changing RX-PRESCALE to 32 by writing 100000 10 in address 2 => reading address 2 of reg file (new UART config) => multiplying 65 and 7 => subtracting with no operands (i.e. 65 - 7) => multiplying 7 and 4 => shifting A right (A should be = 7 = 0000 0111) => **END**

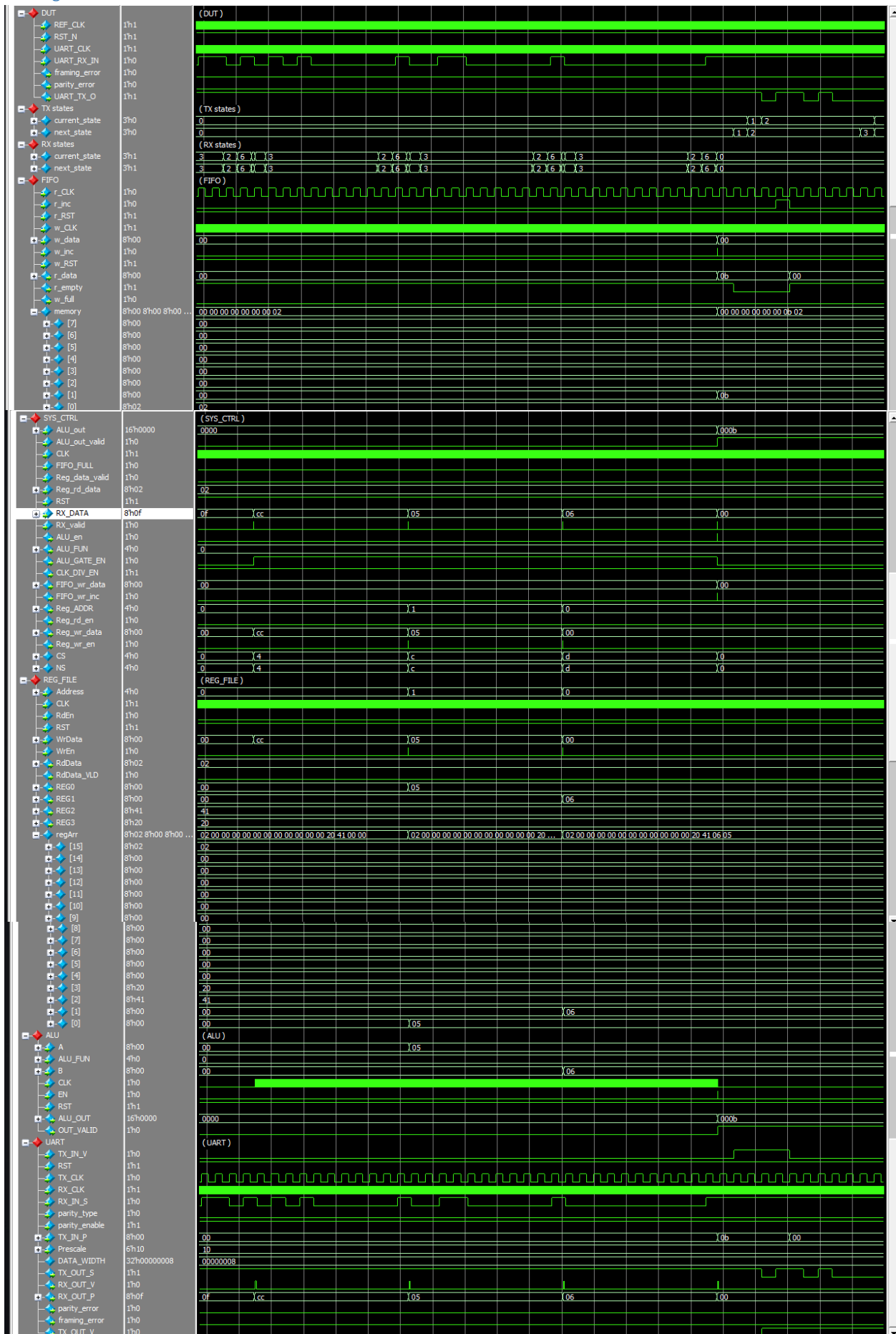
Writing 02 in address F of reg file:



reading address F of reg file:



adding 5 and 6:



The screenshot displays a logic analyzer interface with two main panes. The left pane lists various digital signals, and the right pane shows their corresponding waveforms over time.

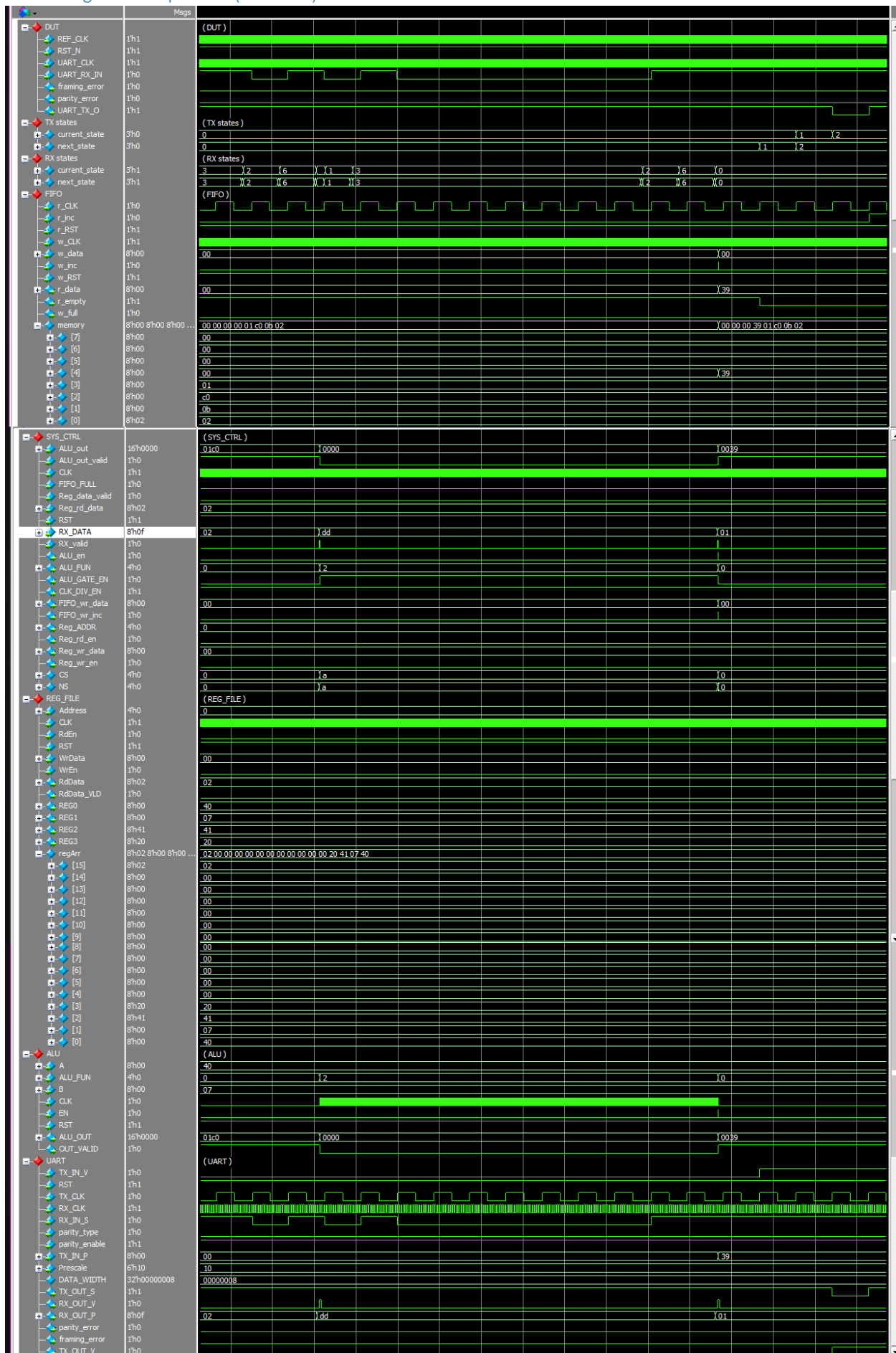
Left Pane (Signal List):

- OUT:** REF_CLK (1h1), RST_CLK (1h1), UART_RX_IN (1h0), framing_error (1h0), parity_error (1h0), UART_TX_O (1h1).
- TX states:** current_state (3h0), next_state (3h0).
- RX states:** current_state (3h1), next_state (3h1).
- FIFO:** r_CLK (1h0), r_inc (1h0), r_RST (1h1), w_CLK (1h1), w_data (8h00), w_inc (1h0), w_RST (1h1), r_data (8h00), r_empty (1h1), w_full (1h0), memory (8h00 8h00 8h00 ...).
- SYS_CTRL:** ALU_out (16h0000), ALU_out_valid (1h0), CLK (1h1), FIFO_FULL (1h0), Reg_data_valid (1h0), Reg_rd_data (8h02), RST (1h1).
- RX_DATA:** RX_valid (8h0f), ALU_en (1h0), ALU_FUN (1h0), ALU_GATE_EN (1h0), CLK_DIV_EN (1h1), FIFO_wr_data (8h00), FIFO_wr_inc (1h0), Reg_ADDR (4h0), Reg_rd_en (1h0), Reg_wr_data (8h00), Reg_wr_en (1h0), CS (4h0), NS (4h0).
- REG_FILE:** Address (4h0), CLK (1h1), RdEn (1h0), RST (1h1), WrData (8h00), WrEn (1h0), RdData (8h02), RdData_VLD (1h0), REG0 (8h00), REG1 (8h00), REG2 (8h41), REG3 (8h20), regArr (8h02 8h00 8h00 ...).
- ALU:** A (8h00), ALU_FUN (4h0), B (8h00), CLK (1h0), EN (1h0), RST (1h1), ALU_OUT (16h0000), OUT_VALID (1h0).
- UART:** TX_IN_V (1h0), RST (1h1), TX_CLK (1h0), RX_CLK (1h1), RX_IN_S (1h0), parity_type (1h0), parity_enable (1h1), TX_IN_P (8h00), Prescale (6h10), DATA_WIDTH (32h00000008), TX_OUT_S (1h1), RX_OUT_V (1h0), RX_OUT_P (8h0f), parity_error (1h0), framing_error (1h0), TX_OUT_V (1h0).

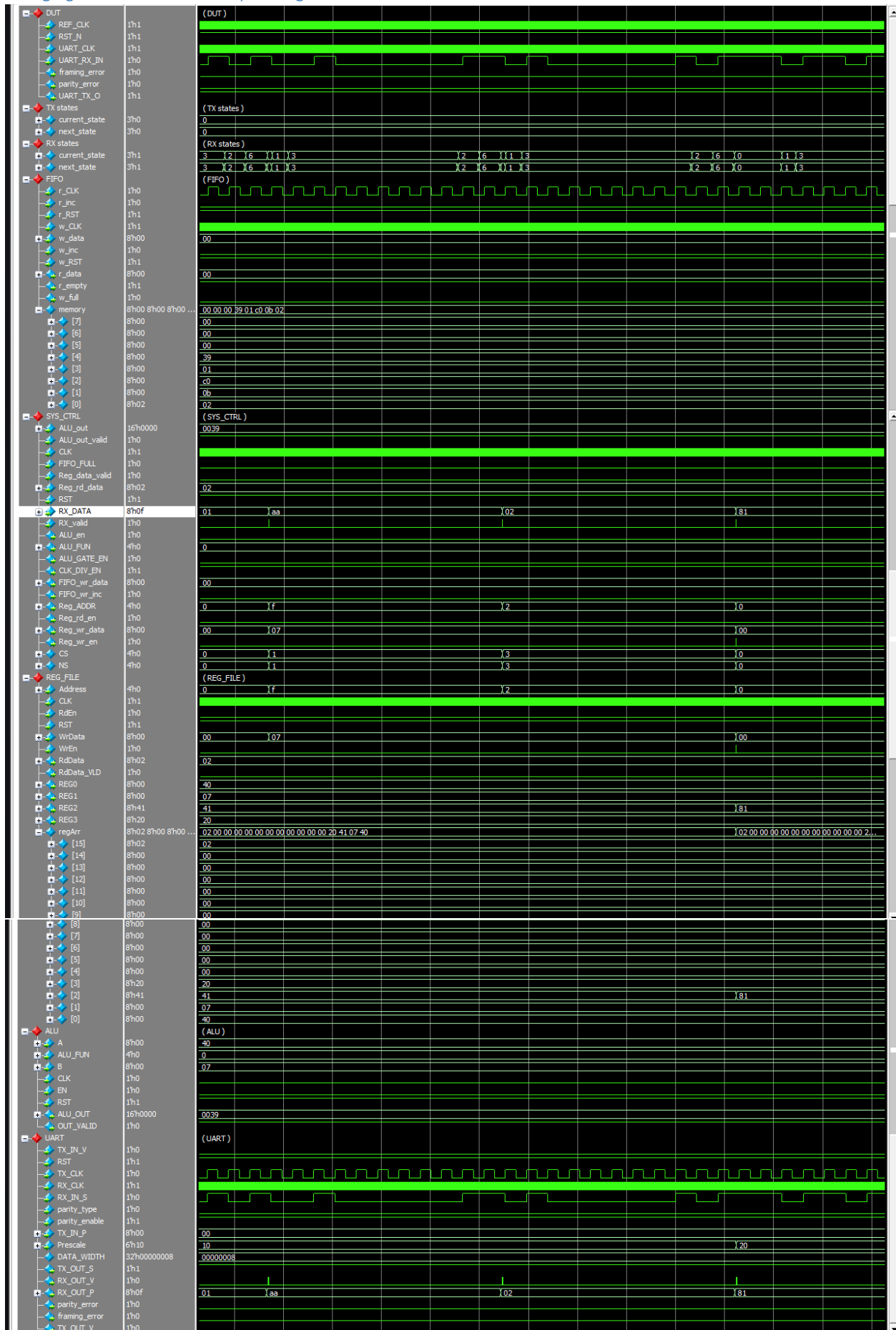
Right Pane (Waveforms):

- (DUT):** Shows digital signals for the Device Under Test, including REF_CLK, RST_CLK, UART_RX_IN, framing_error, parity_error, UART_TX_O, and various state and data signals.
- (TX states):** Shows TX current_state and next_state.
- (RX states):** Shows RX current_state and next_state.
- (FIFO):** Shows FIFO read and write signals.
- (SYS_CTRL):** Shows system control signals like ALU_out, ALU_out_valid, CLK, FIFO_FULL, Reg_data_valid, Reg_rd_data, and RST.
- (RX_DATA):** Shows RX data signals like RX_valid, ALU_en, ALU_FUN, ALU_GATE_EN, CLK_DIV_EN, FIFO_wr_data, FIFO_wr_inc, Reg_ADDR, Reg_rd_en, Reg_wr_data, Reg_wr_en, CS, and NS.
- (REG_FILE):** Shows register file signals like Address, CLK, RdEn, RST, WrData, WrEn, RdData, RdData_VLD, and various registers (REG0, REG1, REG2, REG3, regArr).
- (ALU):** Shows ALU signals like A, ALU_FUN, B, CLK, EN, RST, ALU_OUT, and OUT_VALID.
- (UART):** Shows UART signals like TX_IN_V, RST, TX_CLK, RX_CLK, RX_IN_S, parity_type, parity_enable, TX_IN_P, Prescale, DATA_WIDTH, TX_OUT_S, RX_OUT_V, RX_OUT_P, parity_error, framing_error, and TX_OUT_V.

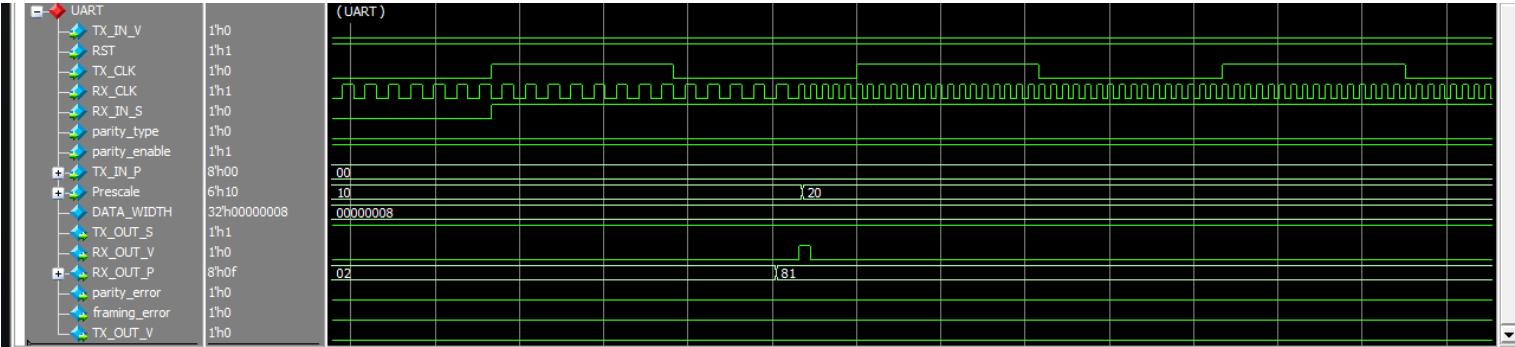
subtracting with no operands (i.e. 64 - 7):



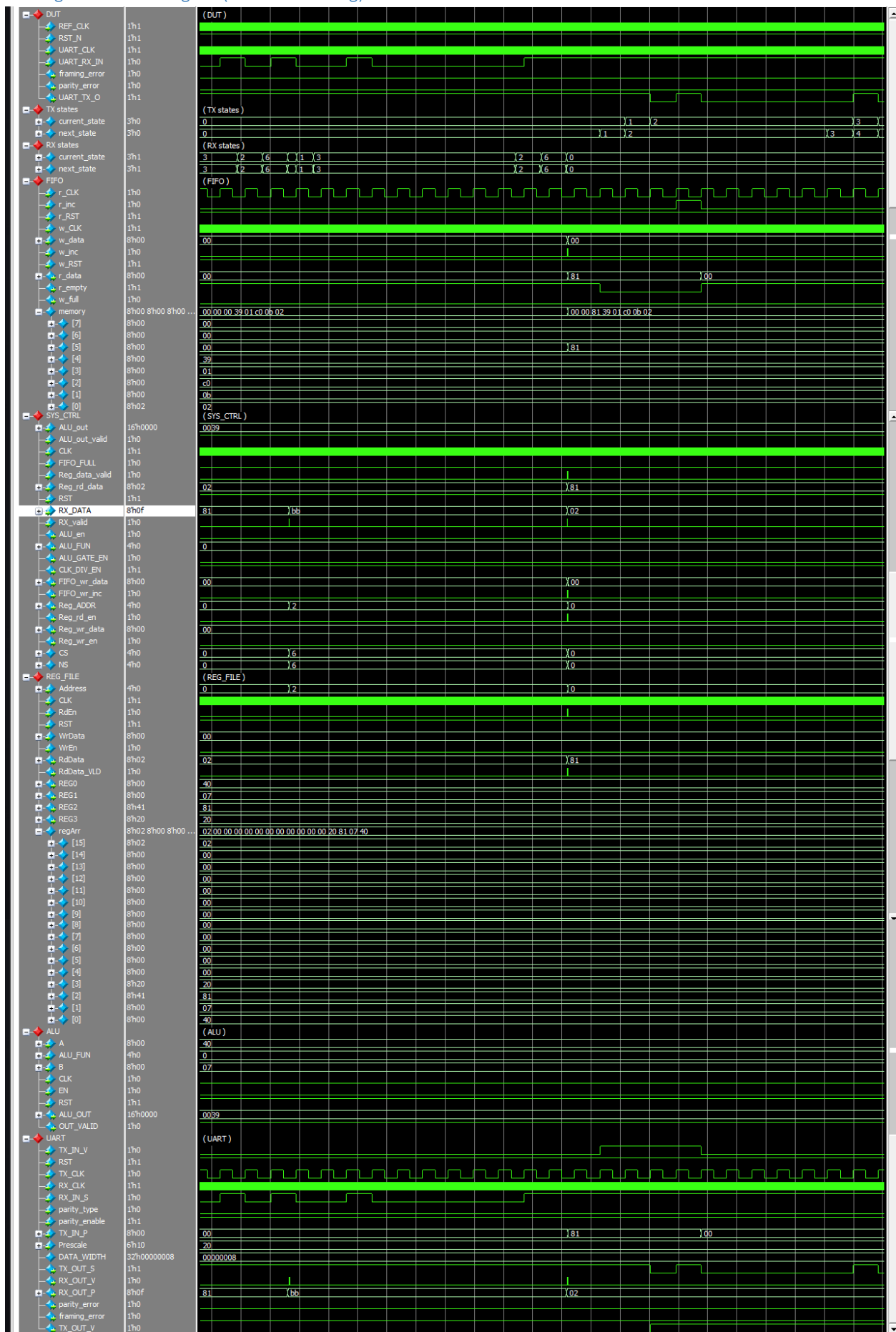
changing RX-PRESCALE to 32 by writing 100000 10:



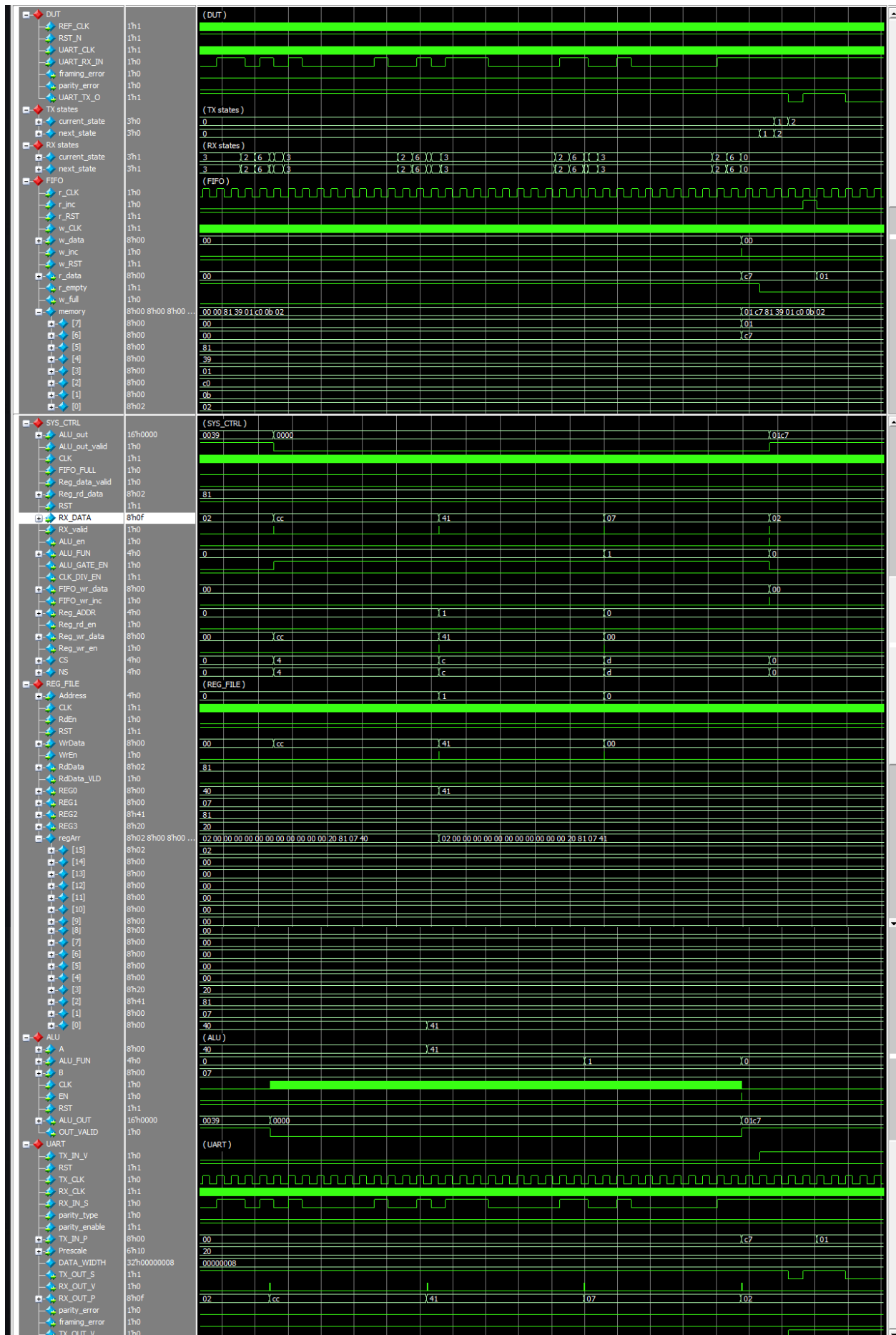
Zooming in on the UART IOs:



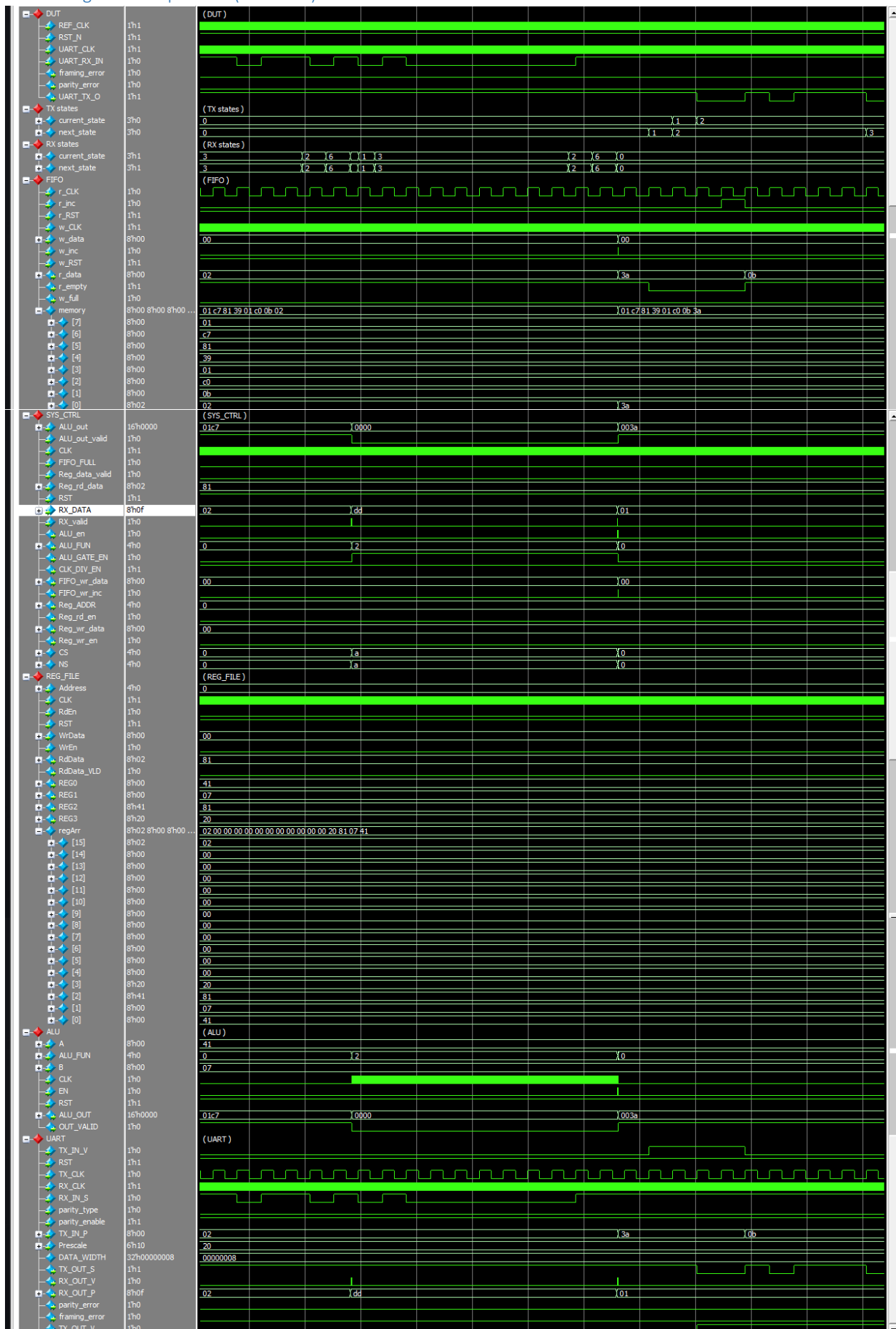
reading address 2 of reg file (new UART config):



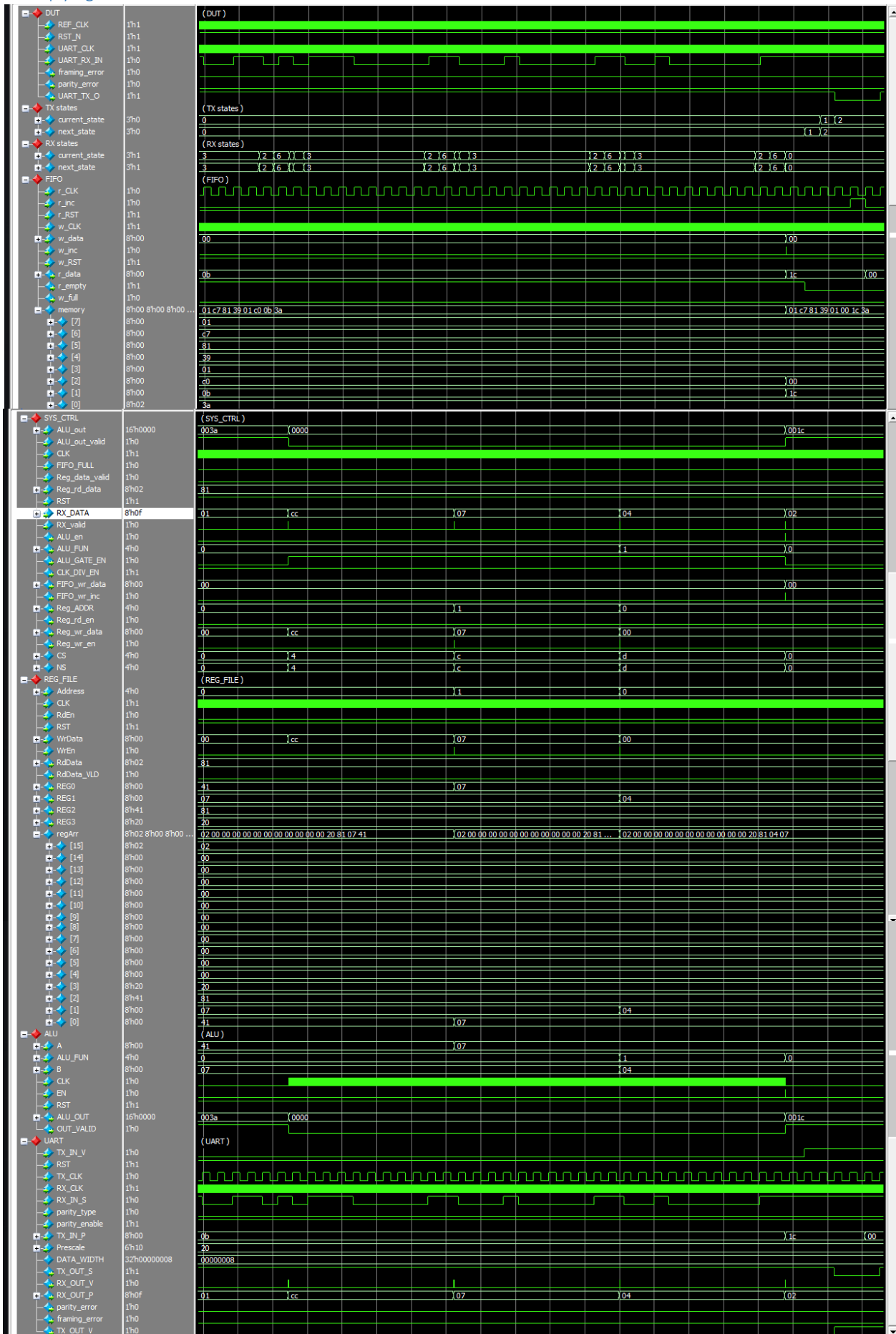
multiplying 65 and 7:



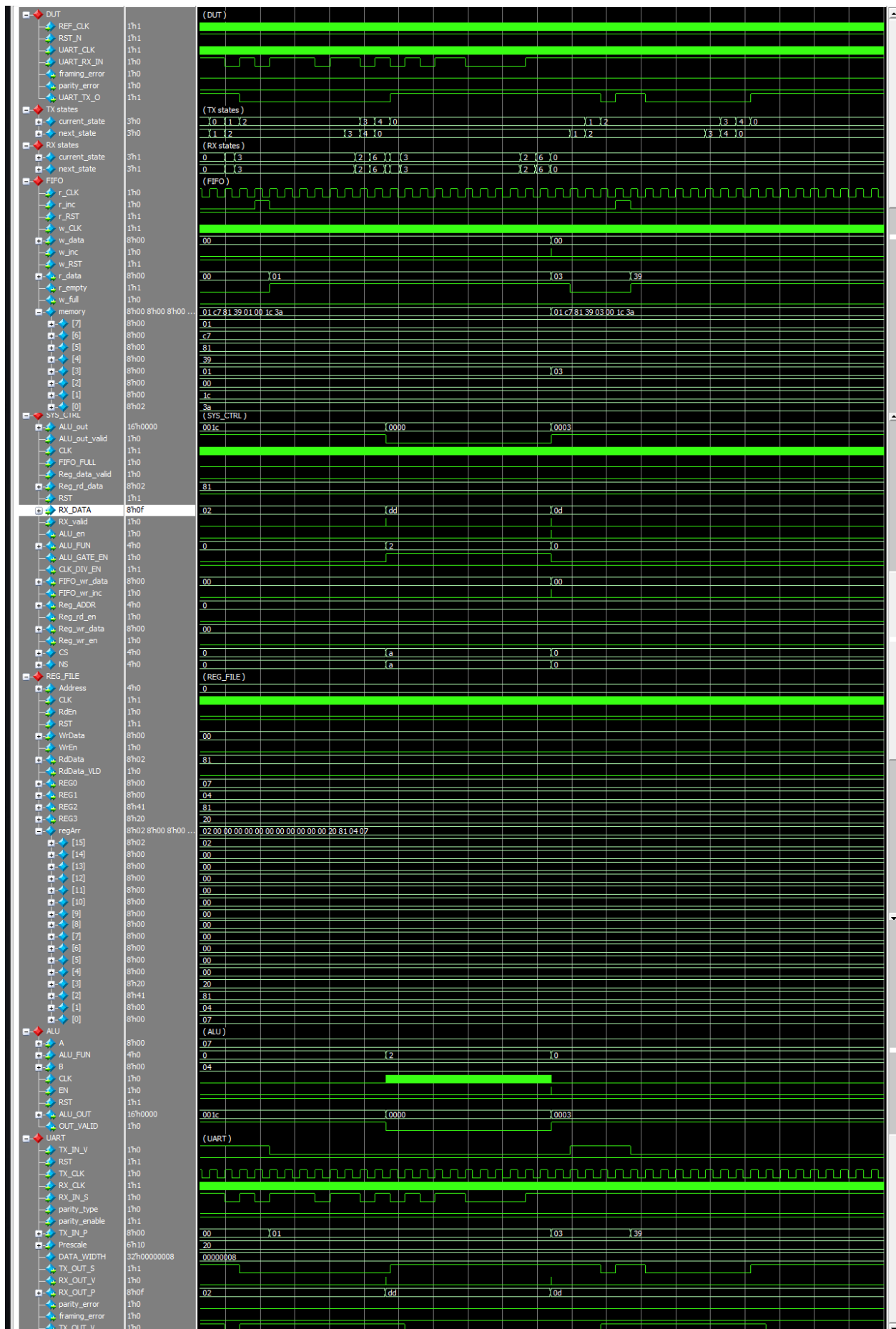
subtracting with no operands (i.e. 65 - 7):



multiplying 7 and 4:



shifting A right (A should be = 7 = 0000 0111):



Transcript:

```
# Test Case 0 : checking initial RX frequency before anything
# test case number 0 : the output frequency is around 1.843318 MHz
# Test Case 1 : writing 2 in ADDRESS F of reg file
# Printing regArr memory array:
# -----
# regArr[0] = 00
# regArr[1] = 00
# regArr[2] = 41
# regArr[3] = 20
# regArr[4] = 00
# regArr[5] = 00
# regArr[6] = 00
# regArr[7] = 00
# regArr[8] = 00
# regArr[9] = 00
# regArr[10] = 00
# regArr[11] = 00
# regArr[12] = 00
# regArr[13] = 00
# regArr[14] = 00
# regArr[15] = 02
# -----
# Test Case 2 : reading data in ADDRESS F of reg file
# Test Case 2 is succeeded result is indeed :2
# Test Case 3 : ADDING 5 and 6
# Test Case 3 is succeeded result is indeed :b
# Test Case 4 : MULTIPLYING 64 and 7
# Test Case 41 is succeeded result is indeed :c0
# Test Case 42 is succeeded result is indeed :1
# Test Case 5 : SUBTRACTING B FROM A (A=64,B=7)
# Test Case 5 is succeeded result is indeed :39
# Test Case 6 : changing UART config to : 100000_01 (prescale = 32,config-register=regArr[2])
# Printing regArr memory array:
# -----
# regArr[0] = 40
# regArr[1] = 07
# regArr[2] = 81
# regArr[3] = 20
# regArr[4] = 00
# regArr[5] = 00
# regArr[6] = 00
# regArr[7] = 00
# regArr[8] = 00
# regArr[9] = 00
# regArr[10] = 00
# regArr[11] = 00
# regArr[12] = 00
# regArr[13] = 00
# regArr[14] = 00
# regArr[15] = 02
# -----
# test case number 6 : the output frequency is around 3.686636 MHz
# Test Case 7 : reading data in ADDRESS 2 of reg file (new UART config)
# Test Case 7 is succeeded result is indeed :81
# Test Case 8 : MULTIPLYING 64 and 7 (A=65,B=7)
# Test Case 81 is succeeded result is indeed :c7
# Test Case 82 is succeeded result is indeed :1
# Test Case 9 : SUBTRACTING B FROM A (A=65,B=7)
# Test Case 9 is succeeded result is indeed :3a
# Test Case 10 : MULTIPLYING 7 and 4 (A=7,B=4)
# Test Case 10 is succeeded result is indeed :1c
# Test Case 11 : SHIFTING A RIGHT (A=7,B=4)
# Test Case 11 is succeeded result is indeed :3
# ** Note: $stop : C:/questasim64_2021.1/examples/SYSTEM_TB.v(104)
# Time: 4557825882564 fs Iteration: 0 Instance: /SYSTEM_tb
```